University of Bonn · Winter Semester 2026/27

Fair Division

Selected Topics in Algorithms and Optimization (Modul V5C4)

Hannaneh Akrami
hakrami@uni-bonn.de
hanoon.github.io/Fair-Division-Course

Lecture 3: Indivisible Items: EF1 and EFX

26 October 2026

Outline of this session. We move from divisible to *indivisible* items. Envy-freeness and proportionality are now too strong, so we introduce the two most famous relaxations of envy-freeness: EFX and EF1. We then focus on EF1 and present three algorithms that compute EF1 allocations: round robin (additive valuations), envy-cycle elimination (monotone valuations), and top-trading envy-cycle elimination (doubly monotone valuations, which include goods and chores).

The Big Picture

Recall the four types of resources from Lecture 1:

	divisible	indivisible
desirable	cake cutting (Lectures 1–2)	**indivisible goods** (this lecture)
undesirable	divisible chores	**indivisible chores** (end of this lecture)

So far we divided a *divisible* cake. Many resources, however, cannot be cut: houses, cars, jobs, courses, or household chores. From now on, the items are *indivisible*.

The model

An instance is a triple $I = (N, M, V)$:

- $N = [n]$ is the set of *agents*;
- M is the set of m indivisible *goods*;
- $V = (v_1, \dots, v_n)$, where $v_i : 2^M \to \mathbb{R}_{\geq 0}$ is the *valuation function* of agent i, with $v_i(\emptyset) = 0$.

For a good g we write $v_i(g)$ instead of $v_i(\{g\})$. We consider two classes of valuations.

- **Monotone:** $v_i(S) \leq v_i(T)$ for all $S \subseteq T \subseteq M$.
- **Additive:** $v_i(S) = \sum_{g \in S} v_i(g)$ for all $S \subseteq M$.

Every additive valuation (with non-negative values) is monotone. An *allocation* is a partition $X = (X_1, \dots, X_n)$ of M, where agent i receives the *bundle* X_i. During our algorithms we also use *partial* allocations, in which some goods are not allocated; an allocation in which every good is allocated is *complete*.

Example 1. Three agents with additive valuations over five goods:

	g_1	g_2	g_3	g_4	g_5
agent 1	4	1	2	2	2
agent 2	1	0	5	1	1
agent 3	1	1	5	1	1

For instance, if $X_1 = \{g_1\}$ and $X_2 = \{g_2, g_5\}$, then $v_1(X_1) = 4$ and $v_1(X_2) = 1 + 2 = 3$.

Envy-freeness and Proportionality Are Too Strong

Recall the two fairness notions from the divisible setting. An allocation X is

- **envy-free (EF)** if $v_i(X_i) \geq v_i(X_j)$ for all agents i, j;
- **proportional (PROP)** if $v_i(X_i) \geq \frac{1}{n}\, v_i(M)$ for all agents i.

For divisible goods, both always exist. With indivisible goods, neither does: consider two agents and a single good g that both value positively. Whoever does not get g receives value 0, so she envies the other agent, and $0 < \frac{1}{2}v_i(g) = \frac{1}{2}v_i(M)$. Hence there is no complete EF allocation and no PROP allocation. We therefore *relax* these notions; in this lecture we focus on relaxations of envy-freeness.

Relaxations of Envy-freeness

Definition 1 (EF1; Lipton, Markakis, Mossel, Saberi 2004; Budish 2011)**.** An allocation X is **envy-free up to one good (EF1)** if for all agents i, j with $X_j \neq \emptyset$ there is a good $g \in X_j$ such that

$$v_i(X_i) \;\geq\; v_i(X_j \setminus \{g\}).$$

In words: any envy of i towards j disappears after removing *some* good from j's bundle. For monotone valuations, if i does not envy j, the condition holds for every $g \in X_j$; so EF1 only constrains pairs with envy.

Definition 2 (EFX; Caragiannis, Kurokawa, Moulin, Procaccia, Shah, Wang 2016)**.** An allocation X is **envy-free up to any good (EFX)** if for all agents i, j and *every* good $g \in X_j$,

$$v_i(X_i) \;\geq\; v_i(X_j \setminus \{g\}).$$

In words: the envy disappears after removing *any* good, even the least valuable one, from the envied bundle. Clearly EF $\Rightarrow$ EFX $\Rightarrow$ EF1 (for monotone valuations).

Example 2. In Example 1, consider $X_1 = \{g_1, g_4\}$, $X_2 = \{g_3, g_5\}$, $X_3 = \{g_2\}$. Agents 1 and 2 envy nobody ($v_1(X_1) = 6 \geq v_1(X_2) = 4$, $v_1(X_3) = 1$; $v_2(X_2) = 6 \geq v_2(X_1) = 2$, $v_2(X_3) = 0$). Agent 3 has $v_3(X_3) = 1$ and envies both others:

- $v_3(X_1) = 2$, and $v_3(X_1 \setminus \{g_1\}) = 1 \leq 1$;
- $v_3(X_2) = 6$, and $v_3(X_2 \setminus \{g_3\}) = 1 \leq 1$, but $v_3(X_2 \setminus \{g_5\}) = 5 > 1$.

So X is EF1, but not EFX: removing the *least* valuable good g_5 from X_2 does not eliminate agent 3's envy.

What is known

- **EF1:** complete EF1 allocations always exist for monotone valuations and can be computed in polynomial time. This is the focus of today's lecture.
- **EFX:** complete EFX allocations need *not* exist for monotone valuations: there are counterexamples with $n \geq 3$ agents, even with submodular valuations (Akrami, Mayorov, Mehlhorn, Srinivas, Weidenbach 2026; Mackenzie, Suzuki 2026). EFX allocations are known to exist for $n = 2$ agents with monotone valuations (Plaut, Roughgarden 2018) and for $n = 3$ agents with additive valuations (Chaudhury, Garg, Mehlhorn 2020). For additive valuations, existence beyond 3 agents is a major open problem.

Round Robin: EF1 for Additive Valuations

Algorithm 1 (Round robin)**.** Fix an order of the agents, say $1, 2, \ldots, n$. The agents pick goods in the order $1, 2, \ldots, n, 1, 2, \ldots, n, 1, \ldots$; at each turn, the current agent picks her favourite good among the remaining ones (ties broken arbitrarily). Stop when no goods are left.

In Example 1, round robin with order $1, 2, 3$ (breaking ties in favour of the smaller index) proceeds as follows: agent 1 picks g_1, agent 2 picks g_3, agent 3 picks g_2 (all remaining goods are worth 1 to her), agent 1 picks g_4 (a tie with g_5), and agent 2 picks g_5. This is exactly the allocation of Example 2; in particular, round robin need not output an EFX allocation.

Theorem 1. *For additive valuations, round robin computes a complete EF1 allocation in polynomial time.*

Proof. The running time is clear. Fix two agents $i \neq j$. Let $g_i^1, g_i^2, \ldots$ and $g_j^1, g_j^2, \ldots$ be the goods picked by i and j, in the order in which they were picked; see Figure 1.

Case 1: i comes before j in the order. For every t, agent i picks g_i^t while g_j^t is still available, so $v_i(g_i^t) \geq v_i(g_j^t)$. Moreover j picks at most as many goods as i. By additivity,

$$v_i(X_i) = \sum_t v_i(g_i^t) \;\geq\; \sum_t v_i(g_j^t) = v_i(X_j),$$

so i does not envy j at all.

Case 2: j comes before i. Remove the first good g_j^1 picked by j. For every t, agent i picks g_i^t while g_j^{t+1} is still available, so $v_i(g_i^t) \geq v_i(g_j^{t+1})$; and j picks at most one more good than i. Hence

$$v_i(X_i) = \sum_t v_i(g_i^t) \;\geq\; \sum_t v_i(g_j^{t+1}) = v_i(X_j \setminus \{g_j^1\}).$$ □

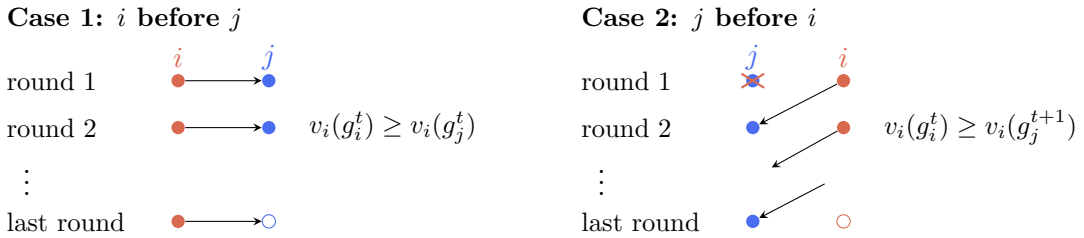


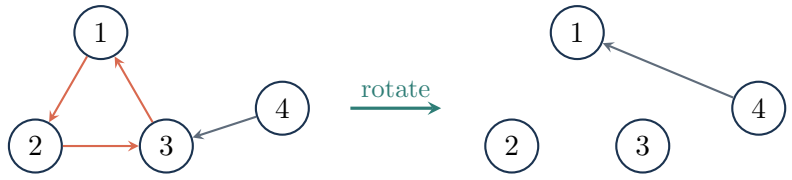
Figure 1: The comparisons in the proof that round robin is EF1. Each row is a round; hollow circles indicate that the agent may not pick in the last round. In Case 2, j's first pick g_j^1 is removed.

The proof uses additivity in an essential way: it compares bundles good by good. For general monotone valuations we need a different idea.

Envy-cycle Elimination: EF1 for Monotone Valuations

Definition 3 (Envy graph)**.** For a (partial) allocation X, the **envy graph** G_X is the directed graph on the agents with an edge $i \to j$ whenever i envies j, i.e. $v_i(X_i) < v_i(X_j)$. An agent with no incoming edge (nobody envies her) is a **source**; an agent with no outgoing edge (she envies nobody) is a **sink**.

If G_X contains a cycle $C = (i_1 \to i_2 \to \cdots \to i_k \to i_1)$, we can **rotate** the bundles along C: every agent i_t on the cycle receives the bundle of the agent i_{t+1} she envies, and all other agents keep their bundles. We denote the new allocation by X^C.



envy cycle $1 \to 2 \to 3 \to 1$ $\qquad$ $1 \leftarrow X_2,\ 2 \leftarrow X_3,\ 3 \leftarrow X_1$

(In the example, agent 4 envied the bundle X_3, which now belongs to agent 1; the cycle agents may also stop envying each other.)

Algorithm 2 (Envy-cycle elimination; Lipton, Markakis, Mossel, Saberi 2004)**.** Start with the empty allocation. Process the goods one by one, in any order. For each good g:

1. While G_X contains a cycle C, rotate the bundles along C.
2. Now G_X is acyclic, so it has a source s. Give g to s.

Why a source? Nobody envies s, so any envy towards s caused by the new good g disappears when g is removed again. This is made precise in Lemma 2 below.

Example 3. We run envy-cycle elimination on Example 1, processing the goods in the order g_3, g_1, g_2, g_4, g_5 and breaking ties between sources in favour of the smaller index.

	Step	X_1	X_2	X_3	**Edges of G_X**
1	$g_3 \to$ agent 1 (all are sources)	$\{g_3\}$	$\emptyset$	$\emptyset$	$2 \to 1,\ 3 \to 1$
2	$g_1 \to$ agent 2 (sources: 2, 3)	$\{g_3\}$	$\{g_1\}$	$\emptyset$	$\mathbf{1 \to 2},\ \mathbf{2 \to 1},\ 3 \to 1,\ 3 \to 2$
3	rotate along $1 \to 2 \to 1$	$\{g_1\}$	$\{g_3\}$	$\emptyset$	$3 \to 1,\ 3 \to 2$
4	$g_2 \to$ agent 3 (only source)	$\{g_1\}$	$\{g_3\}$	$\{g_2\}$	$3 \to 2$
5	$g_4 \to$ agent 1 (sources: 1, 3)	$\{g_1, g_4\}$	$\{g_3\}$	$\{g_2\}$	$3 \to 1,\ 3 \to 2$
6	$g_5 \to$ agent 3 (only source)	$\{g_1, g_4\}$	$\{g_3\}$	$\{g_2, g_5\}$	$3 \to 2$

After step 2, agent 1 prefers g_1 (value 4) to her own g_3 (value 2), and agent 2 prefers g_3 (value 5) to her own g_1 (value 1), so $1 \to 2 \to 1$ is an envy cycle; the rotation in step 3 swaps the two bundles and makes both agents happier. In the final allocation, agent 3 envies agent 2 ($v_3(X_3) = 2 < 5 = v_3(X_2)$), but $v_3(X_2 \setminus \{g_3\}) = 0 \le 2$; all other agents envy nobody. So the output is EF1.

Lemma 1. *Let C be a cycle in G_X. Then*

(a) every agent weakly prefers her bundle in X^C to her bundle in X, and agents on C strictly prefer it;

(b) if X is EF1, then so is X^C;

(c) G_{X^C} has strictly fewer edges than G_X.

Proof. (a) holds by construction. Note that X and X^C consist of the same bundles, only their owners changed.

(b) Fix an agent i and a bundle B of X^C owned by another agent. If B was i's own bundle in X, then $v_i(B) = v_i(X_i) \le v_i(X_i^C)$ by (a), so i does not envy B. Otherwise B was owned by another agent in X too, so by EF1 of X there is $g \in B$ with $v_i(B \setminus \{g\}) \le v_i(X_i) \le v_i(X_i^C)$.

(c) An agent i envies a bundle B if and only if $v_i(B) > v_i(\text{own bundle})$. For an agent off the cycle, her bundle and the set of all bundles are unchanged, so she envies the same bundles as

before: her out-degree is unchanged. An agent i on the cycle strictly increases her value; every bundle she envies in X^C she also envied in X, and she envied in X the bundle she now holds, which she does not envy anymore. So her out-degree drops by at least one. □

Lemma 2. *Let X be an EF1 partial allocation, let s be a source of G_X, and let g be an unallocated good. Then giving g to s yields an EF1 allocation.*

Proof. Only s's bundle changes, to $X_s \cup \{g\}$. By monotonicity, s is not worse off, so EF1 from s's point of view is preserved. For any other agent i, since nobody envies s, $v_i(X_i) \geq v_i(X_s) = v_i\big((X_s \cup \{g\}) \setminus \{g\}\big)$. □

Theorem 2 (Lipton, Markakis, Mossel, Saberi 2004)**.** *For monotone valuations, envy-cycle elimination computes a complete EF1 allocation in polynomial time.*

Proof. The empty allocation is EF1, and by Lemmas 1(b) and 2 every step preserves EF1. A source exists because a finite acyclic graph has a vertex without incoming edges (walk backwards along edges; since there is no cycle, the walk must stop). By Lemma 1(c), each good causes at most $n(n-1)$ rotations, and cycles can be found in polynomial time. Hence the algorithm runs in time polynomial in n and m (given oracle access to the valuations). □

Note that additivity was not used anywhere: the proof only needs monotonicity and value queries $v_i(S)$.

Goods and Chores: Top-trading Envy-cycle Elimination

Doubly monotone valuations

Now items may be *goods* for some agents and *chores* for others. Valuations may take negative values: $v_i : 2^M \to \mathbb{R}$, and M is now a set of *items*.

Definition 4 (Doubly monotone; Bhaskar, Sricharan, Vaish 2021)**.** The valuation v_i is **doubly monotone** if M can be partitioned as $M = G_i \uplus C_i$ such that for all $S \subseteq M$ and $o \notin S$:

$$v_i(S \cup \{o\}) \geq v_i(S) \text{ if } o \in G_i \text{ (a } \textit{good} \text{ for } i), \qquad v_i(S \cup \{o\}) \leq v_i(S) \text{ if } o \in C_i \text{ (a } \textit{chore} \text{ for } i).$$

Monotone valuations over goods ($G_i = M$) are doubly monotone, and so are monotone *chores* ($C_i = M$), and additive valuations with positive and negative values. An item can be a good for one agent and a chore for another.

Definition 5 (EF1 for goods and chores)**.** An allocation X is **EF1** if for all agents i, j, either $v_i(X_i) \geq v_i(X_j)$, or there is an item $o \in X_i \cup X_j$ with

$$v_i(X_i \setminus \{o\}) \;\geq\; v_i(X_j \setminus \{o\}).$$

So the envy can be removed either by removing an item (a good) from the envied bundle, or by removing an item (a chore) from one's own bundle. For goods only, this is the same as before: removing a good from one's own bundle never helps.

Why envy-cycle elimination does not suffice

For a chore, the natural analogue of Lemma 2 is to give it to a *sink*, i.e. an agent who envies nobody: she can then remove the new chore from her own bundle. A sink may not exist; then every agent has an outgoing edge, so G_X has a cycle, which we would like to rotate. However, rotating an *arbitrary* envy cycle can destroy EF1: an agent on the cycle gets a bundle she likes more, but may lose her EF1 "witness" (a chore in her old bundle whose removal used to eliminate her envy).

Example 4. Three agents with additive valuations over four chores:

	c_1	c_2	c_3	c_4
agent 1	-10	-3	-3	-1
agent 2	-1	-1	-5	-1
agent 3	-1	-1	-1	-1

The allocation $X_1 = \{c_1\}$, $X_2 = \{c_2, c_3\}$, $X_3 = \{c_4\}$ is EF1: agent 1 envies agents 2 and 3, but $v_1(X_1 \setminus \{c_1\}) = 0$ beats both; agent 2 envies agents 1 and 3, but $v_2(X_2 \setminus \{c_3\}) = -1 \geq -1$; agent 3 envies nobody. Since agent 1 envies agent 2 and agent 2 envies agent 1, $1 \to 2 \to 1$ is an envy cycle. After rotating it, $X_1' = \{c_2, c_3\}$ and $X_2' = \{c_1\}$. Now agent 1 envies agent 3 ($-6 < -1$), and removing c_2, c_3 or c_4 gives $-3 < -1$, $-3 < -1$ and $-6 < 0$, respectively: the new allocation is *not* EF1. Note that this cycle is not a "top-trading" cycle: agent 1's favourite bundle is X_3, not X_2.

The fix is to rotate only such special cycles.

Definition 6 (Top-trading envy graph)**.** The **top-trading envy graph** T_X has an edge $i \to j$ if X_j is a *most preferred* bundle of agent i and i envies j:

$$v_i(X_j) = \max_{k \in N} v_i(X_k) \quad \text{and} \quad v_i(X_j) > v_i(X_i).$$

T_X is a subgraph of the envy graph G_X. After rotating a cycle of T_X, every agent on the cycle holds one of her most preferred bundles, so she envies nobody: all agents on the cycle become sinks.

The algorithm

Call an item *objective chore* if it is a chore for every agent, and otherwise (if it is a good for at least one agent) a *goods-type item*. For a goods-type item o, let $N_o = \{i \in N : \ o \in G_i\} \neq \emptyset$.

Algorithm 3 (Top-trading envy-cycle elimination; Bhaskar, Sricharan, Vaish 2021)**.** Start with the empty allocation.

Phase 1 (goods-type items). For each goods-type item o:

(a) while G_X contains a cycle, rotate the bundles along it;

(b) choose a source s of the subgraph $G_X[N_o]$ induced by N_o (an agent who regards o as a good and whom no agent in N_o envies), and give o to s.

Phase 2 (objective chores). For each objective chore c:

(a) if G_X has no sink, rotate the bundles along a cycle of the top-trading envy graph T_X;

(b) give c to a sink of G_X.

For goods only, Phase 2 is empty and Phase 1 is envy-cycle elimination; for chores only, Phase 1 is empty.

Analysis of Phase 1

In Phase 1 we maintain a stronger invariant: for all agents i, j,

$$(*) \qquad v_i(X_i) \geq v_i(X_j) \quad \text{or} \quad \exists\, g \in X_j : \ v_i(X_i) \geq v_i(X_j \setminus \{g\}).$$

Clearly $(*)$ implies EF1 (take $o = g$). In words, during Phase 1 envy can always be removed from the *envied* bundle; this is why rotating arbitrary cycles is harmless here.

Lemma 3. *Rotating a cycle of G_X and step (b) of Phase 1 preserve $(*)$. Moreover, a source of $G_X[N_o]$ exists in step (b).*

Proof. Rotation. Agents on the cycle strictly gain and the others keep their bundle; the set of bundles is unchanged. Fix i and a bundle B of another agent in the new allocation. If B is i's old bundle, i does not envy it. Otherwise, if i envies B now, she also envied it before (her value did not decrease), so by $(*)$ there was $g \in B$ with $v_i(B \setminus \{g\}) \leq v_i(\text{old bundle}) \leq v_i(\text{new bundle})$.

Adding o to s. Only s's bundle changes, to $X_s \cup \{o\}$. Since $o \in G_s$, agent s does not lose value, and $(*)$ for s is preserved as in the rotation case. Now let $i \neq s$.

- If $i \in N_o$, then i does not envy s (as s is a source of $G_X[N_o]$), so $v_i(X_i) \geq v_i(X_s) = v_i\big((X_s \cup \{o\}) \setminus \{o\}\big)$.
- If $i \notin N_o$, then o is a chore for i, so $v_i(X_s \cup \{o\}) \leq v_i(X_s)$. If i envies $X_s \cup \{o\}$, she also envied X_s, so by $(*)$ there is $g \in X_s$ with $v_i(X_i) \geq v_i(X_s \setminus \{g\}) \geq v_i\big((X_s \setminus \{g\}) \cup \{o\}\big)$.

Existence. After step (a), G_X is acyclic, hence so is the induced subgraph $G_X[N_o]$; since $N_o \neq \emptyset$, it has a source. □

As in Lemma 1(c), every rotation decreases the number of edges of G_X (that proof only compared values), so Phase 1 runs in polynomial time.

Analysis of Phase 2

Lemma 4. *(a) If G_X has no sink, then T_X contains a cycle. After rotating it, every agent on the cycle is a sink.*

(b) Rotating a cycle of T_X preserves EF1.

(c) Giving an objective chore c to a sink k of G_X preserves EF1.

Proof. (a) If no agent is a sink, every agent i envies someone, so a most preferred bundle of i belongs to another agent and is strictly better for i than her own: i has out-degree at least 1 in T_X. Following out-edges from any agent, we must revisit an agent, which gives a cycle. After the rotation, each agent on the cycle holds one of her most preferred bundles, so she envies nobody.

(b) Agents on the cycle envy nobody afterwards. Every other agent keeps her bundle, and the bundles of the others are the same sets as before (only their owners changed); since the EF1 condition between i and j only depends on the sets X_i and X_j, it still holds.

(c) Only k's bundle changes, to $X_k \cup \{c\}$. Agent k envied nobody, so for every j: $v_k\big((X_k \cup \{c\}) \setminus \{c\}\big) = v_k(X_k) \geq v_k(X_j) = v_k(X_j \setminus \{c\})$. For $i \neq k$: c is a chore for i, so $v_i(X_k \cup \{c\}) \leq v_i(X_k)$. If i envies $X_k \cup \{c\}$, she also envied X_k, so by EF1 there is $o \in X_i \cup X_k$ with $v_i(X_i \setminus \{o\}) \geq v_i(X_k \setminus \{o\}) \geq v_i\big((X_k \cup \{c\}) \setminus \{o\}\big)$. □

Compare (b) with Example 4: there, agent 1 was on the rotated cycle but did not receive a most preferred bundle, so she still envied someone after the rotation and had lost her witness c_1.

Theorem 3 (Bhaskar, Sricharan, Vaish 2021)**.** *For doubly monotone valuations, top-trading envy-cycle elimination computes a complete EF1 allocation in polynomial time.*

Proof. By Lemma 3, the allocation after Phase 1 satisfies $(*)$, hence is EF1. By Lemma 4, Phase 2 keeps it EF1, and each objective chore requires at most one rotation, which creates a sink. Cycles in G_X and T_X can be found in polynomial time. □

Summary.

Algorithm	Valuations	Guarantee
Round robin	additive goods	EF1
Envy-cycle elimination	monotone goods	EF1
Top-trading envy-cycle elimination	doubly monotone (goods and chores)	EF1

Exercises

Question 1 (EFX for identical orderings). Suppose all agents have additive valuations and the *same preference order* over the goods: there is an ordering $g_1, g_2, \ldots, g_m$ of M such that $v_i(g_1) \geq v_i(g_2) \geq \cdots \geq v_i(g_m)$ for every agent i. Prove that a complete EFX allocation exists and can be computed in polynomial time.

Question 2 (EFX for two agents). Prove that for two agents with additive valuations, a complete EFX allocation always exists and can be computed in polynomial time.